

Group assignment 1

Ben Bünnagel, Finn Hetzler, Nils Rudolf, Rena Schaller

MSc. Business Analytics

2615: Web and Cloud Computing

Nova School of Business and Economics

March 2025

1 Introduction

Job search is increasingly important, especially for soon-to-be Masters graduates approaching their first full-time employment. But often, tracking the availability of different job offers is a time-consuming and tedious task, managing different job platforms, with job offers sometimes appearing on multiple platforms, sometimes just on a single one.

Therefore, maintaining an overview of job offers is not always straightforward and requires constant monitoring. After receiving similar input on job search from our peers, we worked in the scope of our course Web and Cloud computing on a combined job search application for Nova students, showing relevant jobs for certain departments across several job platforms, with the option to search for specific job offers, while providing further information about the accurate description for the specific job types and analytics on the current job offer situation. In this group project, we were given the task of developing a web application that uses data from public application programming interfaces (APIs) to provide job offer monitoring as a service to a user, and we want to use this project as a platform to provide a meaningful contribution to the Nova community.

In the following, we will discuss our project, including the used data sources and APIs, the used methodology and system architecture, as well as our implementation roadmap and limitations of the project.

2 Data source and API analysis

2.1 JobSpy Repository Overview

The main source we want to use to scrape multiple job platforms for relevant job listings is a Python library called JobSpy. JobSpy is open-source and freely available at GitHub [1]. It is able to scrape jobs from LinkedIn, Indeed, Glassdoor, Google & ZipRecruiter and present results for search queries regarding jobs on these platforms in a CSV or Excel format.

By default, in the output file, the following scraped information for each job is shown:

site from which job is scraped, job URL, direct URL of job, job title, company name, location, date the job was posted, job type, salary information, remote/not remote, job level, job function, job description, company information plus a few more fields specific to some of the scraped websites.

When searching for a job with JobSpy, the following optional parameters can be set to define the search: site from which job is scraped, search term, location, distance, job type, proxies the scraper will round robin through to avoid IP banning for excessive scraping, remote/not remote, max nr. of results for each site, nr. of hours since job was posted and some more specific filters for some of the scraped websites.

2.2 API and API Details

JobSpy itself is not an API but it leverages the APIs of both Glassdoor [2] and Indeed [3] to get job data via GraphQL. Meanwhile, for Google Jobs and LinkedIn raw HTML is scraped. Since we plan on using the Python library itself, we do not connect to any APIs by using JobSpy directly.

To load in data from JobSpy via an API, we generated a csv file with outputs of jobs from JobSpy, uploaded it to Google Sheets and then used the Google Sheets API to access this data. The Google Sheets API [4] is a RESTful interface that lets you access a spreadsheet's data in Python. It was perfect for our use case since it is free, can store up to 2-MB max of data and has a rate limit of 300 requests per minute per project, more than enough for our purpose of loading a few hundred jobs.

Finally, we used a subset of the EMSI Lightcast Skills API [5], an open-source and free collection of over 32 000 skills from real-world use cases. We used this API to get matching job descriptions for common job roles in the realm of Business Analytics such as “Data Scientist” or “Data Engineer”, allowing our website users to understand what these jobs entail.

2.3 Scraping challenges

The biggest potential issue we face is the site's protections in place against excessive scraping or using of their APIs. This may result in issues of IP addresses being banned, requiring rotating proxies to ensure data is always scrapable when needed. More in-

formation on our exact system architecture, including how we plan on mitigating this problem, can be seen in the following section.

3 Project methodology and system architecture

Our web application is designed to provide a tailored job search experience for German Business Analytics students, aggregating job listings from multiple portals and offering advanced search capabilities along with insightful analytics. The system leverages a three-layer architecture to ensure clarity, maintainability, and scalability. This section provides a comprehensive overview of the system’s architecture, detailing its key components and highlighting non-obvious aspects of its functionality.

3.1 Methodology

Our project methodology aimed around delivering an MVP specifically designed for Business Analytics students, with a modular structure that supports future growth as we plan to continue this project. To start, we narrowed our focus to German job listings for a few key reasons. First, German students make up the largest demographic in Business Analytics at NOVA. Second, we are Germans, and concentrating on the domestic market made the initial development process and data collection efforts more efficient.

For this purpose, we first created a master data template that can be connected in the future to a live data pipeline and a more efficient database. As a result, we scraped the data once and loaded it into Google Sheets and the local job description database.

During the development process, we conducted research to identify the most effective AI tools for web development. While there are several models adept at coding and chatbots specialized for front-end and back-end design, our expectations were not fully met by these solutions. As a result, we focused on using Anthropic’s Claude 3.7 model, which is currently regarded as one of the leading models in coding. However, we encountered challenges with its handling of front-end and back-end communication, which did not perform as expected. We had to manually code these components ourselves.

3.2 System architecture

The application follows a modular three-layer architecture, separating data access, business logic, and presentation:

Data Access Layer (data.py)

The data access layer manages direct interactions with data sources and handles low-level data operations. It interacts with the Google Sheets API to retrieve job listings from a Spreadsheet. We also use `pytest` for testing and `Pydantic` with mock data for data validation to ensure the reliability of our API module. Additionally, it maintains a local SQLite database for storing job skill descriptions, enabling quick retrieval of static content. This layer also manages dynamic search functionality by implementing a relevance scoring system that prioritizes exact matches, followed by synonym matches, and then fuzzy matches.

Business Logic Layer (logic.py)

The business logic layer contains the core application logic, orchestrating data processing and query handling. It processes user search queries, applies validation rules, and transforms data for presentation. Furthermore, it prepares data for the analytics dashboard, summarizing job market trends and providing valuable insights for users.

Presentation Layer (app.py)

The presentation layer handles user interactions, renders interfaces, and manages HTTP requests. Built using `Flask`, this layer defines routes for the job search page, analytics dashboard, and API endpoints. It renders HTML templates for the user interface and returns Dictionary responses for API requests.

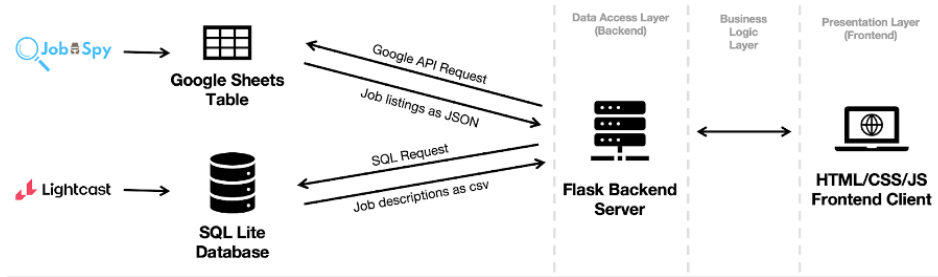


Figure 1: Visual representation of the system architecture

3.3 Data Flow example

To illustrate the system's workflow, consider the following example:

1. A user submits a search query (e.g., "Data Scientist") via the search input interface.
2. The presentation layer (app.py) receives the request and invokes the search_jobs() function in the business logic layer (logic.py).
3. The business logic layer processes the query and calls the data access layer (data.py) to fetch job listings from Google Sheets.
4. The data access layer applies fuzzy matching and synonym handling, retrieving and ranking relevant listings.
5. 20 job results are passed back through the layers where the business logic layer formats the data, and the presentation layer renders the job listings and associated skill descriptions from the SQLite database.
6. When reaching the end of the job list, 20 new jobs are fetched and loaded into the presentation layer. This helps maintain performance.

4 Roadmap and Learnings

The implementation was distributed among our group members as follows:

- Nils Rudolf: Backend, Google sheets API
- Rena Schaller: Analytics, Landing page
- Ben Bünnagel & Finn Hetzler: SQL query, Results page, Routing, API mashup

We began by selecting JobSpy as the primary data source for our project. Additionally, we initially considered incorporating a skills-API to display relevant skills associated with each job, but after further research, we decided to use the Lightcast API to provide more detailed job descriptions. We changed our initial plan because we had this idea of overall job descriptions being especially helpful for students entering the job market. One of the primary challenges was ensuring that the search term entered by the user would be transmitted to the results page, where the relevant data from the Google Sheets API and the Lightcast job description API would be displayed. We needed to ensure that the search query was passed correctly through the URL. This required working on the routing and the use of JavaScript to manage content loading based on the user's search. One of the most unexpected challenges was dealing with small but time-consuming CSS changes on the results page. These seemingly minor adjustments took longer than anticipated, but using a central styles.css file definitely made it easier to ensure consistent styling. A crucial learning point for the project was the importance of maintaining a structured folder architecture and applying the three-layered architecture. The clear structure helped us to better understand the development steps and keep a structure for the project.

Despite the technical challenges, one of the most rewarding aspects of the project was seeing the search functionality and interactive job listings work as intended. After all, we realize that job scraping raises both ethical and legal concerns, particularly regarding the violation of platform terms of service. Because of this, we do advise that our platform be used solely for private, non-commercial purposes.

5 Conclusion

Concluding, we can say that our second phase of the project was successful, as we implemented all our proposed main functionalities, while adhering to the projects criteria. Focusing on the usability of our application for our peers, we implemented an innovative solution to help making job search easier, adding value to our academic community in Business Analytics.

Using Flask as our developing environment for our backend in Python, we access two APIs, storing data obtained from the LightCost API locally in a SQL database in the case of our job descriptions, and storing job listings from JobSpy in our own developed API based on Google Sheets, fetching current job offers by directly placing API calls to our sheets API. We created a dynamic user interface that allows the user to invoke job queries based on search terms, as well as enabling interactions with the displayed data and visualizations, using HTML, CSS and Javascript.

Furthermore, as our idea is being picked up by the Nova Tech Club, we are happy to see interest in our project from fellow students beyond the scope of this course and look ahead to see our idea mature in the near future and become available for more students.

Bibliography

- [1] Github - cullenwatson/jobspy: Jobs scraper library for linkedin, indeed, glassdoor, google, bayt and ziprecruiter. Accessed February 23, 2025. URL: <https://github.com/cullenwatson/JobSpy>.
- [2] Glassdoor api documentation — glassdoor. Accessed February 23, 2025. URL: <https://www.glassdoor.com/developer/index.htm>.
- [3] Indeed api documentation. Accessed February 23, 2025. URL: <https://apis.indeed.com/>.
- [4] Google sheets api documentation. Accessed March 16, 2025. URL: <https://developers.google.com/sheets/api/guides/concepts>.
- [5] Lightcast open skills. Accessed March 16, 2025. URL: <https://lightcast.io/open-skills>.

Appendix: User Interface

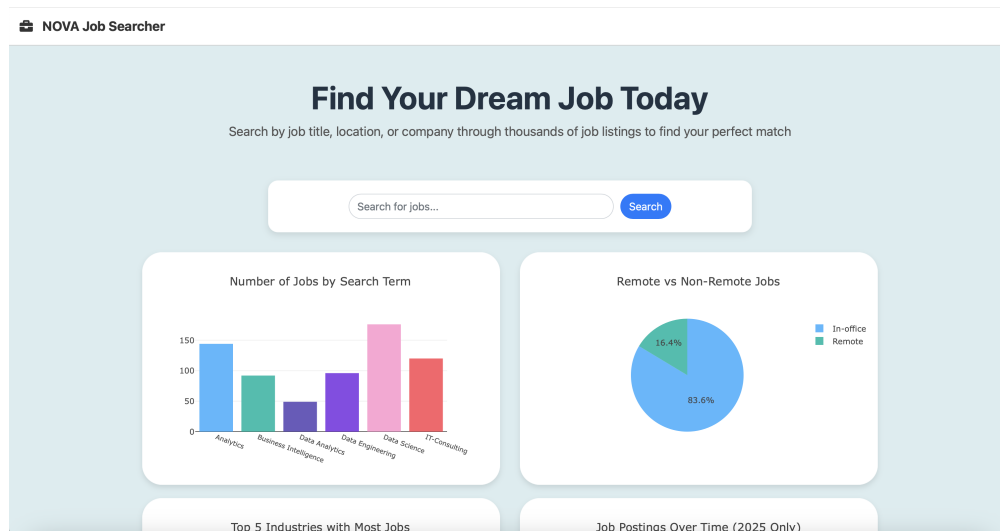


Figure 2: Landing page user interface

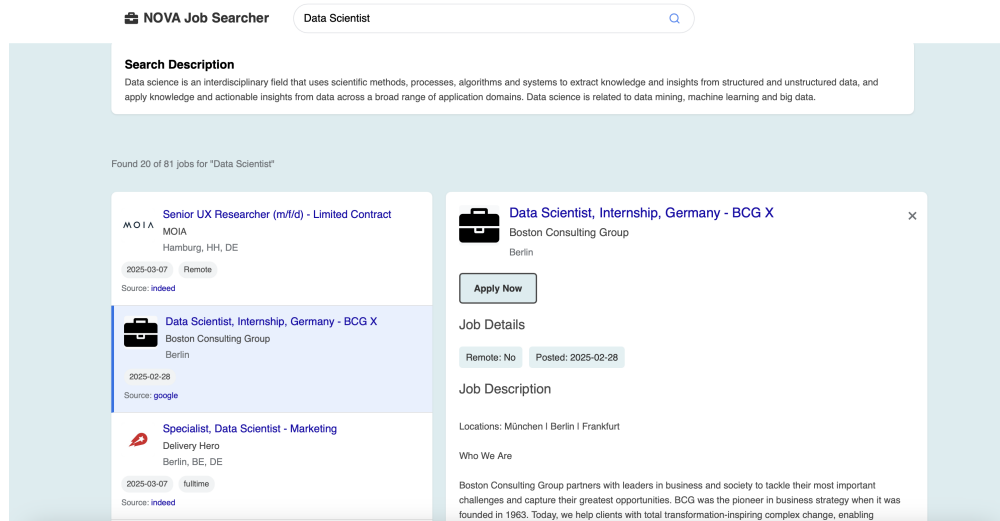


Figure 3: Results page user interface

Appendix: AI use (Chat-GPT & Claude 3.7)

User Prompt: I have the following issue: When I enter a search_term in my starting page and hit search, then in my results page the search term is not passed, only when I enter another search term in the results page, the results for that search term are displayed. So the search term does not get passed from the landing page in the beginning...why could that be? The route is: @app.route('/api/jobs', methods=['GET'])

```
def api_jobs():  
  
    # Get query parameters  
  
    search_term = request.args.get('search_term', '')  
  
    page = int(request.args.get('page', 1))  
  
    limit = int(request.args.get('limit', 20))
```

Generated Solution:

Issue: Search Term Not Passed from Landing Page to Results Page

The issue is that when you enter a search term on the starting page and hit search, the results page does not receive the search term, but when you search again on the results page, it works correctly.

Possible Causes and Fixes:

1. JavaScript Not Passing Search Term Correctly:

Since you're using JavaScript (scripts.js) to make an API call to /api/jobs, ensure that the search term is actually being included in the request when the form is submitted. Your JavaScript should be constructing the URL correctly.

✅ Fix: Check fetch Call in JavaScript

Modify the part where the search request is being made in scripts.js:

```
const queryParams = new URLSearchParams({  
  
    search_term: currentState.searchTerm, // Ensure this has a value  
  
    page: currentState.currentPage,  
  
    limit: currentState.limit  
  
});  
  
const searchTerm = encodeURIComponent(currentState.searchTerm);  
  
const response = await fetch(`/api/jobs?${queryParams}`);
```

💡 Make sure currentState.searchTerm is properly set before calling the API. If this value is "" (empty), it will return all jobs.

2. HTML Form Not Submitting the Search Term Properly:

If you're using an HTML form to submit a search term (instead of JavaScript dynamically handling it), make sure the form is correctly passing the value to the URL.

✅ Fix: Ensure the Form Action Points to the Correct Route

Modify the form in index.html to correctly pass the search term:

Search

◆ Note the method="get" – This will ensure the search term appears in the URL as ?search_term=Data+Science.

3. Flask Route Not Extracting Query Parameters:

Your /api/jobs route should read the search term from the query parameters.

✅ Fix: Ensure Flask Reads search_term Correctly

Your Flask route:

```
@app.route('/api/jobs', methods=['GET'])
```

```
def api_jobs():
```

```
# Get query parameters from the URL
```

```
search_term = request.args.get('search_term', '').strip() # Strip whitespace to avoid empty searches
```

```
page = int(request.args.get('page', 1))
```

```
limit = int(request.args.get('limit', 20))
```

◆ Make sure request.args.get('search_term', '') is actually retrieving the term. If it's empty, that means the form might not be sending it correctly.

4. Redirecting Without Passing Query Parameter:

If you're using redirect(url_for(...)) somewhere in your Flask routes, make sure you include the search term.

✅ Fix: Include Search Term When Redirecting

If you are redirecting to /jobs/, pass the search term as a URL parameter:

```
return redirect(url_for('index_by_city', search_term=search_term))
```

◆ If the term is missing in index_by_city, then the request wasn't correctly redirected.

Summary of Fixes:

1. Ensure JavaScript (scripts.js) correctly appends search_term in the request.
2. Ensure the search form in index.html properly submits search_term.
3. Ensure Flask extracts search_term using request.args.get('search_term', "").
4. If redirecting, pass search_term explicitly in url_for().

Let me know if you need additional debugging steps! 🚀

User prompt:

Develop a responsive job search interface using HTML, CSS, and JavaScript that mimics Google Jobs. The page should include:

1. A fixed header with a shadow effect after scrolling.
2. A dynamic search bar that fetches relevant job listings.
3. A scrollable left-side job list with smooth scrolling.
4. A right-side sliding panel that displays detailed job information when clicking on a job item.
5. Dynamic loading of additional jobs when reaching the bottom of the page.
6. Company logos fetched from URLs (handling missing logos).
7. Mobile-friendly layout with a clean, modern design.

Technical Requirements

File Structure

- HTML: Located in frontend/templates/
- CSS & JS: Stored in frontend/static/

Page Structure

Header

- A search bar for job searches.
- The header should be fixed with a soft shadow after scrolling.

Job Listings Panel (Left Side, Scrollable)

- A list of job results, dynamically loaded in chunks when scrolling.
- Smooth scrolling experience.
- Clicking on a job slides in a detail panel.

Job Detail Panel (Right Side, Sliding In)

- Opens when a job listing is clicked.
- Displays job details, with smooth sliding animation.
- Is scrollable if the content is long.

Dynamic Data Loading

- Fetch more jobs when reaching the page bottom.

- Uses lazy loading to improve performance.

Handling Data

- Job links should prioritize job_url_direct. If missing, use job_url.
- Company logos should be fetched dynamically, with a fallback image if missing.
- Data attributes:
 - site: str
 - job_url: str # always existing
 - job_url_direct: str # sometimes existing, use as primary link
 - title: str
 - company: str
 - location: str
 - date_posted: Optional[str] = None
 - job_type: Optional[str] = None
 - is_remote: Optional[bool] = None
 - job_level: Optional[str] = None
 - job_function: Optional[str] = None
 - listing_type: Optional[str] = None
 - emails: Optional[str] = None
 - description: Optional[str] = None
 - company_industry: Optional[str] = None
 - company_logo: Optional[str] = None # fetched dynamically

Design & UX

- Soft box shadows for elements.
- Smooth animations for list scrolling and detail panel sliding.
- Minimalist UI, similar to Google Jobs.
- Dark mode support (optional but preferred).

Expected Technologies

- Frontend: HTML, CSS (flexbox/grid), JavaScript (vanilla or a lightweight framework like Alpine.js).
- AJAX/Fetch API: For dynamic job fetching without reloading the page.
- CSS Animations: For smooth sliding and scrolling effects.

Accessing Backend Data in a Frontend Project

To access job information (job["title"]) and job descriptions from the database (job_descriptions SQL table), follow these steps:

Fetching Job Data from Google Sheets API

- The Flask backend integrates with Google Sheets using GoogleSheetsClient.
- The endpoint / (GET & POST) queries job data by a search term.
- Example: `result = sheet_client.query_by_search_term(search_term) jobs = pd.DataFrame([job.dict for job in result.jobs]).to_dict(orient='records')`
- This converts job data into a JSON-compatible format.
- The frontend receives this JSON object with jobs.

Fetching Job Descriptions from SQLite

- The backend also connects to an SQLite database (job_descriptions.db).
- SQL query to fetch descriptions:
- `query = "SELECT description FROM job_descriptions WHERE job_title LIKE ?"`
- `cursor.execute(query, ('%' + search_term + '%',))`
- `descriptions = cursor.fetchall()`

Sending Data to the Frontend

- The backend renders test_index.html, passing:
- jobs → List of job dictionaries (title, company, etc.)
- description → First matching job description (or “No matching job descriptions found.”)
- `response = make_response(render_template('test_index.html', jobs=jobs, search_term=search_term, description=description))`

Fetching Data in the Frontend

- The frontend can retrieve job information by making an HTTP request (AJAX, Fetch API, or a frontend framework).

- Example Fetch API call:
- `fetch('/jobs/Data Science')`
- `.then(response => response.json())`
- `.then(data => console.log(data));`
- This fetches job listings and displays them in the UI.

[app.py code]